THE RUST-NATIVE REACT RUNTIME

AlB'DO

Write the React you already know. Ship almost no JavaScript.

A compiler that proves which parts of your app are alive — and ships only those. The rest arrives as finished light.

INVESTOR BRIEF · CONFIDENTIAL

/ albedo — the fraction of light a surface reflects /

THE PROBLEM

The web learned to ship the *engine*, not just the car

Every modern React/Next app builds your UI twice — once as HTML on the server, then again as a megabyte of JavaScript that re-runs in the browser to "hydrate" what was already drawn. Users pay for it in seconds; companies pay for it in cloud bills.

Paying twice for one page

- Render on server
- Send HTML
- Re-send the whole framework
- Re-run it to "hydrate"

~90%

of shipped JavaScript is framework & component code the visible page never needed.

2–6s

to interactive on real mobile devices — hydration blocks the main thread, frame by frame.

+1% lost

in conversion for every 100ms of added latency. Slowness is a line item.

THE SOLUTION

AlB’DO ships the *app* — never the engine

A Rust-native compiler reads your React and proves, component by component, what is truly static, what is data-bound, and what is genuinely interactive. Static and data-bound parts ship as finished HTML. Only real interactivity ships code — surgically.

Same code

Author in ordinary React/TSX. No new mental model, no annotations, no rewrite.

Provably correct

If it can’t prove the cheaper path is safe, it falls back. A broken UI is impossible.

A fraction of the weight

Most pages reach interactive with kilobytes — or zero — of client JavaScript.

BYTES TO INTERACTIVE — TYPICAL CONTENT PAGE

CORE FEATURE · 01

Correct *by construction*

At the heart of AIB'DO is a soundness lattice — a static dataflow analysis that, for every component, computes the complete set of provably-correct ways to ship it. The framework only ever chooses from inside that set.

A — Tier A — Pure

Provably static. Renders to plain HTML. Zero client code, forever.

B — Tier B — Bound

Server-rendered & data-bound. Updates by surgical patch, not by re-running the app.

C — Tier C — Live

Genuinely interactive. Gets a real runtime — and only this tier does.

The line we never cross

A learned model may rank options. It may never invent one. The analysis is inviolable — so the worst a wrong choice can cost is a few milliseconds, never a broken app.

SILENT-WRONG IS IMPOSSIBLE.

CORE FEATURE · 02

An engine built for *this decade*

Underneath the elegance is a Rust core engineered to disappear: a per-request arena allocator with no garbage collector, embedded QuickJS for server rendering, and a near-zero client runtime. It is the fastest path from request to pixels.

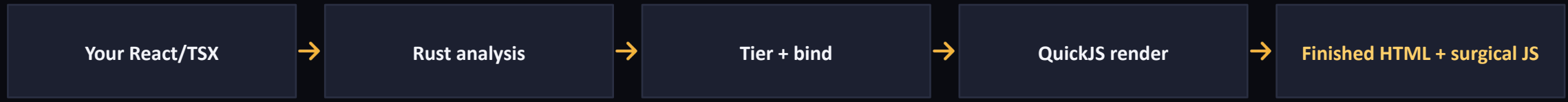
0–18 KB
client JS on a typical page

<1 ms
cold first render, warm pool

~0.24 ms
server action round-trip, p50

Zero
GC pauses — arena reset per request

ONE PASS, REQUEST TO PIXELS



IMPACT · ENTERPRISE

At scale, the savings *compound*

Less JavaScript on the wire is less compute to render, less egress to bill, fewer servers to run, and faster pages that convert. The same product, on a materially smaller infrastructure footprint.

Up to 60%

lower render & edge compute spend

70–95%

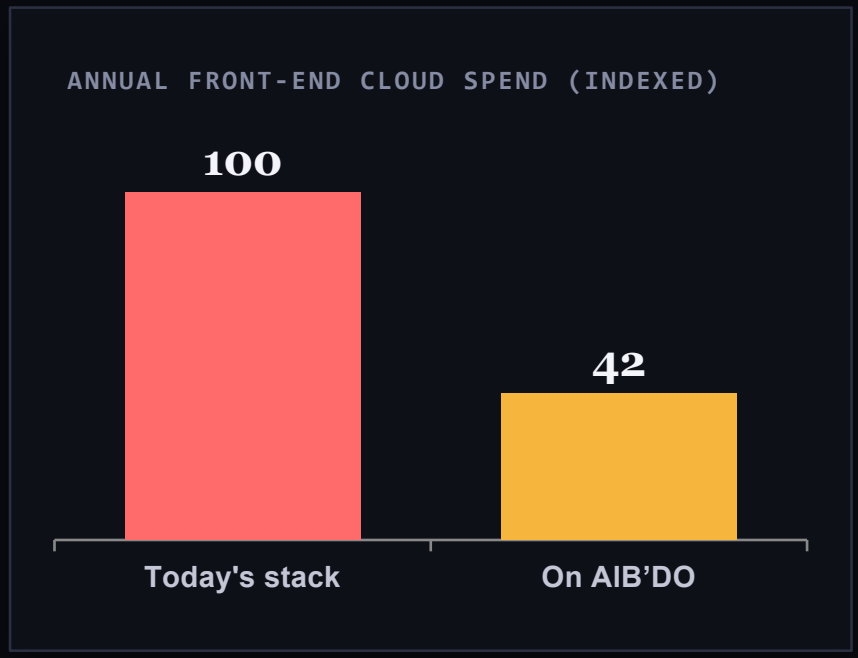
less client JavaScript egress

Fewer servers

headroom reclaimed across the fleet

Lower carbon

less compute is less energy, measurably



IMPACT · INDIE & SOLO BUILDERS

One developer. *Production-grade by default.*

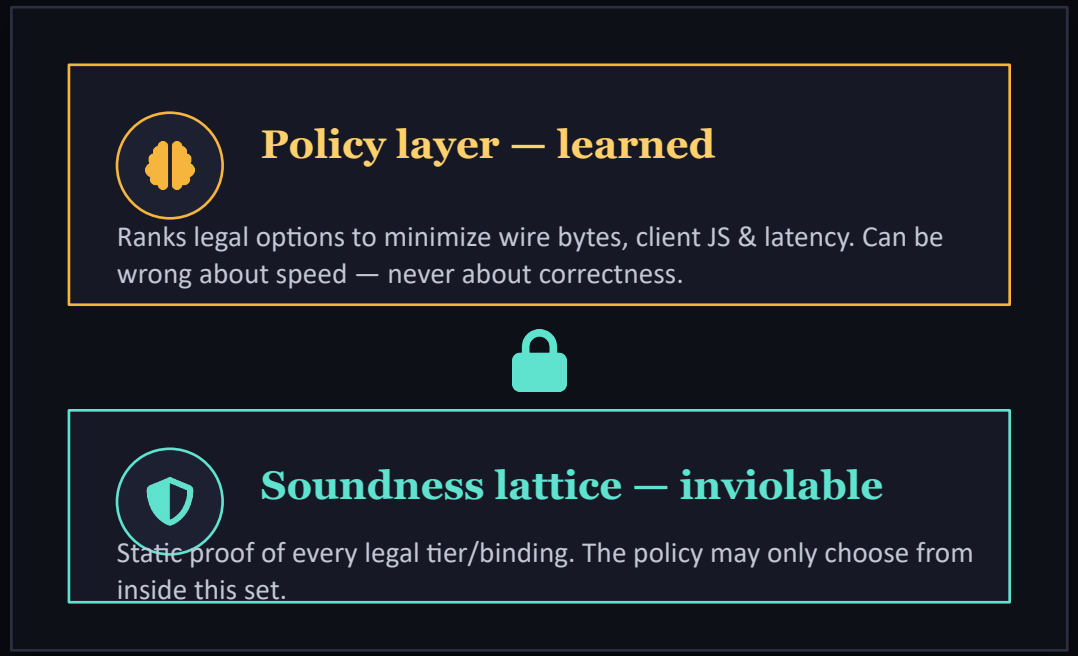
The performance work that used to need a platform team is now the framework's job. You write features; AIB'DO makes them fast. The gap between a side project and a serious product disappears.

Just write React	No perf budget rituals, no manual code-splitting, no "use client" archaeology. Ship the feature.
Elite performance, free	Your weekend project loads like a flagship app — because the compiler does the hard part.
Scale without a rewrite	From first user to viral spike, the same code holds. No re-architecture tax when you grow.
Hosting that stays cheap	Tiny payloads and a lean runtime mean a real product can live on a hobby-tier bill.

THE MOAT · SELF-LEARNING

A framework that *learns your fleet*

AIB'DO separates correctness from optimization. The static analysis decides what is legal. A learned policy — the same architecture that powers ML-guided compilers — chooses the best legal option, and it gets sharper with every page you ship.



The flywheel

Ship Every build emits the choices the policy made.

Measure Real render, bytes & latency stream back as signal.

Learn The policy re-ranks — fleet-wide, across every app.

Improve Tomorrow's build is faster than today's. Automatically.

The advantage AIB'DO accrues — better perf per watt, per build — competitors can't copy. It's earned from data they don't have.

WHERE THIS GOES

From a faster framework to the *web's substrate*

I

The runtime

Rust core, soundness engine & near-zero client runtime — the fastest React you can write today.

II

The learning loop

Fleet-scale policy that compounds: every app makes every other app faster.

III

Universal components

Any component on npm — ShadCN, your own, AI-generated — runs natively, optimally, untouched.

IV

Edge-native streaming

Columnar wire format & WebTransport: state streams to the edge, instantly, anywhere.

V

The default for AI-built apps

When machines write the web, AIB'DO is the substrate that makes it correct and fast — by construction.

Each phase widens the market — and deepens a moat the data keeps refilling.

THE WEB'S NEXT DEFAULT

Write less.
Ship light.

AIB'DO turns ordinary React into the fastest, leanest, provably-correct web — and learns from every page it serves. The framework gets better while you sleep.

Let's build the substrate. →

bishalsenpersonal@gmail.com